7-Day Master Prompt Guide

Project: LLM-Driven Cloud-Native Deployment Benchmarking

Use one module per day. Each day is self-contained: paste the **Master Prompt** into Claude Code (or this chat) at the start of the session, run the **verify commands** at the end, and don't move to the next day until the acceptance checklist passes.

Keep a `project/` folder as your root. Every day's module lives in its own subfolder so nothing collides.

Shared context block

Paste this once at the top of every day's session, then paste that day's Master Prompt right after it.

```
PROJECT CONTEXT (paste at the start of every session):

I'm building "From Repository to Running Service" — a benchmarking system that
evaluates whether LLMs can autonomously analyze a code repo and produce a
working
cloud deployment (Dockerfile + Terraform IaC), then validates the result for
functional correctness, security, and cost.

Stack (all free): Python 3.11, Ollama (local LLM) or Groq free API, Docker,
Terraform, Kind (local Kubernetes), Trivy, Checkov, Infracost, Locust, SQLite,
Streamlit. No paid cloud resources — everything runs locally except one
Oracle Cloud Always Free VM used for a single real-cloud demo run later.

Repo root: project/
Today we are building ONLY the module described below. Assume nothing else
exists yet except what's listed under "Depends on." Write clean, typed,
testable Python. Include a __main__ block or CLI so I can run the module
standalone from the terminal. Include docstrings explaining WHY, not just what.
```

Day 1 — Module 1: Repo Analyzer Agent

Goal: Given a repo path/URL, produce a structured JSON description of its stack (language, framework, entrypoint, port, dependencies, statefulness).

Depends on: nothing (first module).

Master Prompt:

```
Build module: project/analyzer/

Requirements:
1. `analyzer/clone.py` — clones a given GitHub URL (or accepts a local path)
   into project/analyzer/_work/<repo_name>/. Use GitPython or subprocess+git.
2. `analyzer/scan.py` — walks the repo tree (skip .git, node_modules, venv)
   and extracts: file tree (max depth 3), contents of manifest files if present
   (requirements.txt, package.json, pom.xml, go.mod, Pipfile, Dockerfile if any,
   .env.example). Cap total extracted text at ~6000 tokens.
3. `analyzer/schema.py` — a Pydantic model `RepoAnalysis` with fields:
   language, framework, entrypoint, port (int, default guess if unknown),
```

```
   database (nullable), package_manager, has_existing_dockerfile (bool),
   notes (list[str] for anything ambiguous the LLM flagged).
4. `analyzer/llm_client.py` — a thin wrapper with two backends selectable by
   env var LLM_BACKEND=ollama|groq:
   - ollama: POST to http://localhost:11434/api/generate with model
     "qwen2.5-coder:7b" (or whatever's pulled), stream=false.
   - groq: POST to https://api.groq.com/openai/v1/chat/completions using
     GROQ_API_KEY from env, model "llama-3.3-70b-versatile".
   System prompt must force JSON-only output matching RepoAnalysis schema —
   no markdown fences, no commentary. Validate the response against the
   Pydantic model; if validation fails, retry once with the validation error
   appended to the prompt asking it to fix the JSON.
5. `analyzer/run.py` — CLI: `python -m analyzer.run <repo_url_or_path>`
   prints the validated RepoAnalysis as pretty JSON and saves it to
   project/analyzer/_work/<repo_name>/analysis.json.

Write a small test repo fixture under project/analyzer/fixtures/inventory-api/
(a minimal Flask app with requirements.txt: flask, psycopg2-binary, gunicorn)
so I can test without network access to a real GitHub repo.
```

Run & verify:

```
# pull a small local model once
ollama pull qwen2.5-coder:7b

cd project
python -m analyzer.run analyzer/fixtures/inventory-api
cat analyzer/_work/inventory-api/analysis.json
```

Acceptance checklist:

- ☐ Runs against the fixture repo without network access (ollama backend)
- ☐ Output validates against `RepoAnalysis` schema every time (run it 3x)
- ☐ Correctly identifies Flask + postgresql + port 5000 for the fixture

---

Day 2 — Module 2: Artifact Generator

Goal: Given `analysis.json`, generate a Dockerfile and Terraform manifest (K8s deployment + service + configmap) via LLM, and validate both are syntactically correct.

Depends on: Day 1's `analysis.json`.

Master Prompt:

```
Build module: project/generator/

Requirements:
1. `generator/prompts.py` — prompt templates that take RepoAnalysis JSON and
   ask the LLM to produce TWO separate artifacts in one response, delimited
   clearly (e.g. "---DOCKERFILE---" / "---TERRAFORM---"):
   a) A minimal, secure Dockerfile (must include a non-root USER directive,
      multi-stage build if the language benefits from it, .dockerignore
      suggestion in a comment).
   b) A Terraform file targeting the `kubernetes` provider (not a cloud
      provider) with: kubernetes_deployment, kubernetes_service (ClusterIP),
      kubernetes_config_map for env vars. Use variables for image name/tag
      and replica count.
```

```
2. `generator/parse.py` — splits the LLM response on the delimiters into two
   strings; raise a clear error if either section is missing or empty.
3. `generator/validate.py`:
   - Dockerfile: run `docker build --check` if available, else shell out to
     hadolint if installed, else just confirm it starts with FROM and has
     no obvious syntax break (fallback heuristic).
   - Terraform: write to a temp dir, run `terraform init -backend=false` and
     `terraform validate`, capture pass/fail + error text.
4. `generator/run.py` — CLI: `python -m generator.run <path_to_analysis.json>`
   writes project/generated/<repo_name>/Dockerfile and main.tf, prints
   validation results for both, and writes a validation_result.json
   (fields: dockerfile_valid, terraform_valid, errors[]).
5. If validation fails, retry generation ONCE with the error text appended
   to the prompt ("your last output failed validation with: <error>, fix it").

This module should be callable standalone but designed to be imported by the
orchestrator we build on Day 6.
```

Run & verify:

```
cd project
python -m generator.run analyzer/_work/inventory-api/analysis.json
cat generated/inventory-api/validation_result.json
docker build -t inventory-api:test generated/inventory-api/
```

Acceptance checklist:

- ☐ Dockerfile builds successfully with `docker build`
- ☐ `terraform validate` passes on the generated main.tf
- ☐ Dockerfile includes a non-root USER line (check this manually — it's your Day-5 security story if it's ever missing)

Day 3 — Module 3: Local Cluster & Provisioner

Goal: Stand up a free local Kubernetes cluster and a wrapper that applies/destroys the generated Terraform against it.

Depends on: Day 2's `generated/<repo>/main.tf`.

Master Prompt:

```
Build module: project/provisioner/

Requirements:
1. `provisioner/cluster.py` — functions to:
   - ensure_kind_cluster(name="benchmark") — checks if a kind cluster with
     that name exists (`kind get clusters`), creates it if not
     (`kind create cluster --name benchmark`).
   - load_image(image_tag, cluster_name) — runs
     `kind load docker-image <tag> --name <cluster>` so locally built images
     are visible inside the cluster without a registry.
2. `provisioner/terraform_runner.py` — wraps subprocess calls to:
   - terraform init (in the generated repo's folder)
   - terraform apply -auto-approve -var="image=<tag>"
   - terraform destroy -auto-approve (for teardown/reset between runs)
   Capture stdout/stderr, return structured result: {success, duration_sec,
   resources_created, raw_log}.
```

```
3. `provisioner/run.py` — CLI:
   `python -m provisioner.run <repo_name> --image <tag> [--destroy]`
   Full flow: ensure cluster → load image → terraform apply → poll
   `kubectl get pods -l app=<repo_name>` until Running or 90s timeout →
   print pod status. `--destroy` flag tears everything down.

Also write project/provisioner/oracle_cloud_notes.md — a short setup guide
(commands only, no fluff) for creating one Always Free ARM VM on Oracle
Cloud and installing Docker + k3s on it, for Day 7's real-cloud demo run.
This file is documentation only, not code to execute today.
```

Run & verify:

```
cd project
docker build -t inventory-api:v1 generated/inventory-api/
python -m provisioner.run inventory-api --image inventory-api:v1
kubectl get pods
python -m provisioner.run inventory-api --destroy
```

Acceptance checklist:

- ☐ Kind cluster comes up from a cold start (`kind get clusters` empty initially)
- ☐ Pod reaches `Running` state within the timeout
- ☐ `--destroy` cleanly removes all resources, cluster is reusable for the next repo

---

Day 4 — Module 4: Deploy & Validate

Goal: Once a pod is running, confirm it actually works — health check + a load smoke test.

Depends on: Day 3's running pod.

Master Prompt:

```
Build module: project/validator/

Requirements:
1. `validator/port_forward.py` — starts `kubectl port-forward svc/<repo_name>
   <local_port>:<container_port>` as a background subprocess, waits until the
   port is accepting connections (retry loop, 20s timeout), returns the
   process handle so it can be terminated later.
2. `validator/health_check.py` — hits a `/health` (or `/` if `/health` 404s)
   endpoint via requests, returns {status_code, latency_ms, passed: bool}.
3. `validator/loadtest.py` — a Locust locustfile.py (or plain k6 script,
   your choice, prefer Locust since it's pure Python) that runs a 30-second
   smoke test at ~10 concurrent users against the forwarded port, and a
   Python wrapper that runs it headlessly and parses the summary stats
   (requests, failures, p95 latency) from Locust's CSV/JSON export.
4. `validator/run.py` — CLI: `python -m validator.run <repo_name> --port
<container_port>`
   Full flow: port-forward → health check → load test → kill port-forward →
   write project/generated/<repo_name>/validation_report.json with all
   results merged.

Handle the case where health check fails: don't crash, record
{passed: false, reason: "..."} and let the pipeline continue — a failed
deploy is itself a valid benchmark result, not an exception to bubble up.
```

Run & verify:

```
cd project
python -m provisioner.run inventory-api --image inventory-api:v1
python -m validator.run inventory-api --port 5000
cat generated/inventory-api/validation_report.json
```

Acceptance checklist:

- ☐ Health check correctly reports pass/fail without crashing the script either way
- ☐ Load test produces a summary with request count + p95 latency
- ☐ A deliberately broken image (wrong CMD) produces a clean `passed: false` report, not a stack trace

---

Day 5 — Module 5: Security & Cost Scanning

Goal: Score the generated artifacts for security posture and estimated cost.

Depends on: Day 2's Dockerfile/main.tf (doesn't need a running pod).

Master Prompt:

```
Build module: project/scanner/

Requirements:
1. `scanner/security.py`:
   - run_trivy_image(image_tag) — shells out to
     `trivy image --format json <tag>`, parses CVE counts by severity.
   - run_checkov(terraform_dir) — shells out to
     `checkov -d <dir> --output json`, parses passed/failed check counts
     and lists failed check IDs + descriptions.
   - compute_security_score(trivy_result, checkov_result) — a 0-100 score:
     start at 100, subtract points per CRITICAL/HIGH CVE and per failed
     Checkov check (weight critical checks higher). Document the exact
     formula in a docstring — you'll need to defend this scoring choice
     to your professor.
2. `scanner/cost.py`:
   - run_infracost(terraform_dir) — shells out to
     `infracost breakdown --path <dir> --format json`, parses estimated
     monthly cost. Note: infracost needs a cloud-provider-shaped Terraform
     to price against — if we're targeting the local `kubernetes` provider,
     either (a) maintain a small mapping table of "if this ran on a
     comparable managed K8s node, cost ≈ $X" as a documented approximation,
     or (b) generate a parallel AWS/GCP-flavored Terraform variant just for
     costing purposes. Implement (a) — simpler, defensible, keep it explicit
     in the output that it's an approximation, not a live cloud quote.
3. `scanner/run.py` — CLI: `python -m scanner.run <repo_name>`
   writes project/generated/<repo_name>/scan_report.json with
   {security_score, cve_summary, failed_checks, cost_estimate_usd_monthly,
   cost_note}.

Install trivy, checkov, and infracost as local binaries (all free, no signup
needed for trivy/checkov; infracost needs a free API key from
https://www.infracost.io/ — note this in a README but don't hardcode a key).
```

Run & verify:

```
# one-time installs (all free)
```

```
brew install trivy checkov infracost   # or apt/curl equivalents on Linux
infracost auth login                    # free API key

cd project
python -m scanner.run inventory-api
cat generated/inventory-api/scan_report.json
```

Acceptance checklist:

- ☐ Trivy and Checkov both run without manual intervention
- ☐ Security score formula is documented and reproducible
- ☐ Cost estimate has a clear "this is an approximation because ..." note attached — don't present it as a real cloud bill

---

Day 6 — Module 6: Orchestrator & Benchmark Runner

Goal: Chain modules 1–5 across many repos and multiple LLM models, store every run, handle failures gracefully.

Depends on: all of Days 1–5.

Master Prompt:

```
Build module: project/orchestrator/

Requirements:
1. `orchestrator/db.py` — SQLite schema (via sqlite3 or SQLAlchemy) with a
   `runs` table: id, repo_name, llm_model, timestamp, analysis_json,
   dockerfile_valid, terraform_valid, deploy_success, health_passed,
   load_test_p95_ms, security_score, cve_critical_count, cost_estimate_usd,
   time_to_deploy_sec, failure_stage (nullable), failure_reason (nullable).
2. `orchestrator/pipeline.py` — `run_pipeline(repo_path, llm_model)` that
   calls analyzer → generator → provisioner → validator → scanner in
   sequence. Wrap EACH stage in try/except: on failure, record
   failure_stage + failure_reason, skip remaining stages, still write a row
   to the DB (a failed run is data, not a crash). Always call
   provisioner teardown in a `finally` block so failed runs don't leave
   the kind cluster dirty for the next one.
3. `orchestrator/config.py` — a list of repos (path or URL) and a list of
   LLM models/backends to test, e.g.:
   REPOS = ["fixtures/inventory-api", "fixtures/node-notes-api", ...]
   MODELS = [("ollama","qwen2.5-coder:7b"), ("groq","llama-3.3-70b-versatile")]
4. `orchestrator/run_batch.py` — CLI: `python -m orchestrator.run_batch`
   loops REPOS × MODELS, calls run_pipeline for each combination, prints a
   live progress line per run, writes everything to benchmark.db.
5. `orchestrator/report.py` — a function that reads benchmark.db and prints
   a summary table (success rate per model, avg security score per model,
   avg cost, most common failure_stage) — this is your results-chapter data.

Add 2 more small fixture repos beyond inventory-api (e.g. a Node/Express
app, a stateless Go API) so the batch has real variety to compare across.
```

Run & verify:

```
cd project
python -m orchestrator.run_batch
python -m orchestrator.report
```

Acceptance checklist:

- ☐ A deliberately failing repo (e.g. empty folder) produces a DB row with `failure_stage` set, not a crashed script
- ☐ Cluster is clean (`kubectl get pods` empty) after the batch finishes
- ☐ `report.py` output is something you'd be comfortable pasting into your report's results section

---

Day 7 — Module 7: Dashboard, Real-Cloud Run & Report

Goal: Visualize results for your professor and prove one run works on real cloud infra, not just local.

Depends on: Day 6's `benchmark.db`.

Master Prompt:

```
Build module: project/dashboard/

Requirements:
1. `dashboard/app.py` — Streamlit app reading project/orchestrator/benchmark.db:
   - Table of all runs, filterable by repo and model.
   - Bar chart: deploy success rate per model.
   - Scatter plot: security_score (x) vs cost_estimate_usd (y), colored by
     model — this is your key "tradeoff" visual.
   - A "failure taxonomy" panel: group failed runs by failure_stage and show
     counts — this is usually the most interesting part of the results for
     a professor, since it shows WHERE LLMs break, not just whether they do.
2. `dashboard/export.py` — generates a one-page Markdown summary (repo count,
   model count, overall success rate, avg security score, avg cost, top 3
   failure reasons) suitable for pasting into your report's abstract/results.

Separately, write project/provisioner/oracle_run.sh — a short script that
SSHes into the Oracle Cloud Always Free VM (set up per Day 3's notes),
installs k3s if not present, and re-runs the provisioner + validator flow
against it instead of kind, to produce one real-cloud data point.

Deploy the Streamlit dashboard itself to Streamlit Community Cloud (free,
just needs a public GitHub repo and a "Deploy" click) so you can hand your
professor a live link instead of a localhost screenshot.
```

Run & verify:

```
cd project
streamlit run dashboard/app.py
# then, from Streamlit Community Cloud (share.streamlit.io), connect your
# GitHub repo and deploy — free, no card required

bash provisioner/oracle_run.sh   # one real-cloud data point
```

Acceptance checklist:

- ☐ Dashboard loads and reflects the batch from Day 6 without manual data wrangling
- ☐ At least one run in the DB has `env=oracle_cloud` (add this column if you didn't on Day 6) proving a real-cloud deployment happened

- ☐ You have a public Streamlit link + the `export.py` summary ready to paste into your report

---

After Day 7

You'll have: a working pipeline, a benchmark database with comparative data across repos and models, a security/cost tradeoff story, a documented failure taxonomy, and a live dashboard link — which covers the abstract's "structured framework... to assess reliability, limitations, and risks" almost sentence for sentence.